Instalador do GITMENU Desktop

GITMENU Desktop V1

Documentação Completa Corporativa — Manual Técnico, Guia do Usuário e Analista

FICHA TÉCNICA CORPORATIVA
Programa: GITMENU Desktop V1
Arquivo Instalador: install_gitdesktopV1.sh
Versão: V1 Oficial — 2026
Autoria: Everaldo Costa Silva — Especialista Linux — Bash/ksh, VMs, Containers, APIs
Linhas de Código: ~3.040 linhas (instalador + core Bash + GUI Python incorporados)
Tamanho: ~157 KB
Compatibilidade: Debian, Ubuntu, Fedora 44, RHEL, CentOS, Slackware, Arch, openSUSE, Alpine, AIX, HP-UX, Oracle UX
Gerenciadores: apt, dnf, yum, pacman, zypper, apk, xbps, slackpkg, brew, lslpp, swlist
Dependências: python3 3.8+, curl 7.60+, jq 1.5+, git 2.20+, zenity, tkinter, coreutils, diffutils, findutils, tar, gzip, base64
Licença: Corporativa — Uso Interno/Externo Autorizado — Ver Seção 15
Segurança: umask 077, chmod 700, EUID!=0, symlink protection, audit.log, rollback.sh, PAT volátil
Data da Documentação: 04/11/2026

1. Visão Geral

O **install_gitdesktopV1.sh** é o instalador autônomo do **GITMENU Desktop V1**. Ele contém todo o código do aplicativo dentro de si (Bash core + GUI Python Tkinter), sem baixar nada da internet durante a instalação. Fluxo: **diagnóstico** → **instalação de dependências** → **backup/rollback** → **criação** → **validação**.

Após instalar, o usuário utiliza um ícone na Área de Trabalho que abre a interface gráfica com 30 tarefas para gerenciar repositórios GitHub sem abrir o navegador, com cores, atalho visual de pastas e exclusão com caminho exato.

2. Arquitetura Completa — Espelha o Script

2.1 Outer Installer (linhas 1-350):
- Valida EUID!=0, HOME físico via pwd -P, bloqueio de symlink em ~/.local, ~/.local/share, ~/.local/state, ~/.local/share/applications.
- Detecta distro via /etc/os-release e gerenciador (apt, dnf, yum, pacman, zypper, apk).
- Funções: say(), fail(), have_tkinter(), version_at_least(), command_version().
- Verifica versões mínimas, instala faltantes via sudo/doas.
- Cria BACKUP_ROOT=~/.local/state/github-manager-backups/STAMP.XXXXXXXX com mktemp -d, chmod 700, copia BASE, Desktop Entry e App Entry.
- Gera rollback.sh com validação rigorosa de caminhos antes de rm -rf.
- Cria BASE, DESKTOP_DIR, APP_ENTRY com mkdir -p e proteção symlink.
- Gera APP_TMP e GUI_TMP com mktemp, chmod 700, e mv atômico para APP e GUI finais.
- Cria .desktop com bash -n + py_compile + desktop-file-validate + chmod 755/644.

2.2 Inner Core — github-manager.sh (linhas 350-2800):
- umask 077, APP_TITLE, cores ANSI (C_RESET, C_BOLD, C_RED, C_GREEN, C_YELLOW, C_BLUE, C_CYAN, C_BG_BLUE).
- Funções de GUI: GUI_LAST_PROMPT, GUI_CANCELLED, printf() sobrescrito para capturar prompts, gui_read() que usa zenity --entry/--file-selection quando GUI_EMBEDDED=0, e protocolo __GUI_PROMPT__/__GUI_DONE__ quando GUI_EMBEDDED=1.
- AUDIT_LOG em XDG_STATE_HOME/github-manager/audit.log com data, usuário e ação.
- API: urlencode(), urlencode_path() por segmento, api() com curl -sS -o response -w http_code, tratamento HTTP 401 (token inválido), 403 (rate limit), 404, 422, 429.
- Helpers: valid_name(), validate_remote_folder() bloqueia //, \, ., .., /, ensure_login(), logout_github().

2.3 GUI Python — github-manager-gui.py (linhas 2800-3040):
- Tkinter, ttk, threading, subprocess. Janela principal com listbox de 30 tarefas, área de log, campo de diretório local atual, botão selecionar anexos.
- Funções: cleanup_gui(), show_login(), show_menu(), run_task(), main_gui(). Comunicação via subprocess Popen do core Bash com --task.

3. Fluxo de Instalação em 4 Etapas

ETAPA 1/4 — Diagnóstico: lê /etc/os-release, uname -m, id -un, detecta PM, lista PACKAGES, verifica python3, curl, jq, git, tar, gzip, zenity, diff, cmp, find, base64, have_tkinter(). Se missing e PRIV (sudo/doas) existe, executa UPDATE e INSTALL.
ETAPA 2/4 — Backup: cria BACKUP com mktemp -d, chmod 700, grava ambiente.txt e pacotes-instalados.txt, copia instalação anterior, gera rollback.sh 700 com trap EXIT que restaura em falha.
ETAPA 3/4 — Criação: mkdir -p BASE, DESKTOP_DIR, applications, mktemp APP_TMP e GUI_TMP, cat <<'GITHUB_MANAGER_APP_EOF_9A71C2' com todo o core e GUI, chmod 700, mv atômico, bash -n e py_compile.
ETAPA 4/4 — Validação e Atalhos: cria GitHub-Manager.desktop em Desktop e applications com Exec, Icon, Terminal=false, chmod 755/644, valida com desktop-file-validate.

4. Segurança — Espelho Completo do Script

- **umask 077** global, **chmod 700** em todos os scripts, tmp e rollback, 755 em diretórios.
- **EUID!=0:** bloqueia sudo/root no instalador.
- **HOME físico:** HOME_PHYSICAL via pwd -P, recusa symlink.
- **Symlink protection:** verifica -L em ~/.local, ~/.local/share, ~/.local/state, ~/.local/share/applications, BASE, DESKTOP_ENTRY, APP_ENTRY, BACKUP_ROOT, TMP_DIR, APP_TMP, GUI_TMP.
- **mktemp + mv atômico:** para backup, tmp e criação de arquivos, evitando TOCTOU.
- **bash -n + py_compile:** valida sintaxe antes de ativar.
- **base64 seguro:** tr -d \r\n para evitar quebras.
- **jq encoding:** urlencode() e urlencode_path() por segmento, evitando injeção.
- **validate_remote_folder():** bloqueia //, \\, ., .., / inicial/final.
- **Sem sobrescrita:** download usa mktemp + ln atômico, upload verifica sha e pergunta s/n, move verifica destino 404 antes.
- **PAT volátil:** GITHUB_USER e GITHUB_TOKEN apenas em variável de sessão, nunca gravados, logout_github() limpa.
- **API segura:** Accept: application/vnd.github+json, Authorization Bearer, X-GitHub-Api-Version, tratamento de rate limit.
- **Audit.log:** registra data, user, ação, detalhe em XDG_STATE_HOME.
- **Rollback validado:** rollback.sh verifica HOME, BASE_PATH, DESKTOP_PATH, APP_ENTRY_PATH, BACKUP_PATH exatos e ausência de symlink antes de rm -rf.

5. Estrutura de Arquivos e Diretórios

- **Instalador:** install_gitdesktopV1.sh (onde o usuário baixa).
- **Após instalar:**
• BASE=~/.local/share/github-manager/
• APP=BASE/github-manager.sh (core Bash)
• GUI=BASE/github-manager-gui.py (interface)
• APP_ENTRY=~/.local/share/applications/GitHub-Manager.desktop
• DESKTOP_ENTRY=~/Desktop/GitHub-Manager.desktop (ou xdg-user-dir DESKTOP)
• BACKUP_ROOT=~/.local/state/github-manager-backups/STAMP.XXXXXXXX/
- original/github-manager, Desktop-GitHub-Manager.desktop, Applications-GitHub-Manager.desktop
- ambiente.txt, pacotes-instalados.txt
- rollback.sh (700)

• AUDIT_LOG=~/.local/state/github-manager/audit.log
• LOCAL_DIR_CONFIG=~/.config/github-manager/current-directory (pasta local atual)
• TMP_DIR=$TMPDIR/github-manager.XXXXXXXX (temporários da sessão, chmod 700, limpo no EXIT)

6. Dependências e Compatibilidade Multi-Distro

- **Distros detectadas:** Debian/Ubuntu (apt), Fedora/RHEL/CentOS (dnf/yum), Arch (pacman), openSUSE (zypper), Alpine (apk), Void (xbps), Slackware (slackpkg), macOS (brew), AIX (lslpp), HP-UX (swlist), Oracle UX.
- **Pacotes:** python3, python3-tk/tkinter, curl, jq, git, coreutils, diffutils, findutils, tar, gzip, zenity.
- **Versões mínimas:** python 3.8+, curl 7.60+, jq 1.5+, git 2.20+.
- **Funções de verificação:** have_tkinter() testa import tkinter, version_at_least() usa sort -V, command_version() extrai versão de cada binário.

7. Interface Gráfica e Cores V1

- **Cores ANSI:** C_RESET, C_BOLD, C_RED, C_GREEN, C_YELLOW, C_BLUE, C_MAGENTA, C_CYAN, C_WHITE, C_BG_BLUE.
- **Função color_echo():** printf com %b para cores.
- **Função show_simple_repo_info(repo, branch, repo_enc):** chama api GET /repos/.../git/trees/{branch}?recursive=1, lista pastas via jq '.tree[]? | select(.type=="tree") | .path' sort -u head 50, exibe com ícone ■ em verde. Usada em todas as telas de movimentação (mover, excluir, enviar, baixar).
- **gui_read():** captura último printf como prompt (GUI_LAST_PROMPT), se GUI_EMBEDDED=1 imprime __GUI_PROMPT__ e lê stdin; se GUI_EMBEDDED=0 usa zenity --entry ou --file-selection.

8. Lista Completa de Funções Internas (Espelho do Script)

Outer: say(), fail(), have_tkinter(), version_at_least(), command_version()
GUI/Core Helpers: color_echo(), show_simple_repo_info(), audit_log(), printf() sobrescrito, gui_read(), cleanup(), valid_name(), urlencode(), urlencode_path(), api(), ensure_login(), logout_github()
Repositório: list_repositories(), create_repository_main(), select_repository(), delete_repository(), rename_repository()
Branch: create_secondary_branch(), delete_secondary_branch(), rename_secondary_branch(), choose_repository_branch()
Pasta: validate_remote_folder(), create_folder_main(), delete_folder_main(), create_folder_secondary_branch(), delete_folder_secondary_branch(), rename_folder_secondary_branch(), remote_file_paths(), list_remote_folder_files(), list_main_repository_files(), list_secondary_branch_files()
Arquivo — Movimentação: move_remote_files(), move_main_repository_files(), move_secondary_branch_files(), move_all_files_bulk(), move_all_repository_folder_files(), move_all_main_branch_files(), move_all_secondary_branch_files(), move_all_branch_flexible(), upload_local_file_to_github(), upload_file(), upload_selected_attachments() — com BRANCH (ENTER=MAIN) + pasta destino (ENTER=RAIZ), validação branch, remote_path = pasta/arquivo
Arquivo — Transferência: download_file() com mktemp + ln atômico, upload_file() com base64 e sha check
Arquivo — Exclusão Precisa V1: delete_remote_file() — monta caminho EXATO pasta/arquivo ou RAIZ/arquivo, sem busca por basename, valida via /contents/{path}?ref=branch, só apaga se existir no caminho exato
Comparação/Busca: compare_local_remote_file(), search_file(), list_local_files(), get_local_base_dir(), change_current_directory()
Terceiros: clone_third_party_repository()
GUI: cleanup_gui(), show_login(), show_menu(), run_task(), main_gui()

9. Matriz das 30 Tarefas

ID	Nome	Tipo	Entrada Exata	Nota V1
1	Comparar local x remoto	Leitura	Nome arquivo local	diff -u com cores
2	Mover arquivo MAIN	Movimentação	Repo, Branch MAIN, Pasta origem (RAIZ=ENTER), arquivo, Pasta destino	Confirmação dupla

3	Mover secundária	Movimentação	Repo, Branch secundária, origem, destino	Com atalho visual
4	Mover todos de pasta (bulk)	Movimentação	Repo, Branch, Origem, Destino	Valida colisão + preflight 404
5	Mover todos MAIN	Movimentação	Repo, Branch MAIN	Move todos da MAIN
6	Mover flexível (qualquer branch)	Movimentação	Repo, Branch qualquer	Flexível
7	Listar repos com pastas	Listagem	—	Mostra repo + branch + pastas via tree
8	Buscar arquivo	Busca	Trecho nome	Varre repos/branches
9	Listar locais	Listagem	—	find maxdepth 1
10	Listar arquivos MAIN	Listagem	Repo, Branch MAIN, Pasta	Lista via remote_file_paths
11	Listar secundária	Listagem	Repo, Branch secundária, Pasta	Idem
12	Excluir repo	Exclusão	Nome repo + confirma s/n	audit_log DELETE_REPO
13	Excluir branch secundária	Exclusão	Repo + branch secundária + DELETAR	Protege MAIN
14	Excluir pasta MAIN	Exclusão	Repo + pasta + DELETAR	Exclui todos arquivos da pasta
15	Excluir pasta secundária	Exclusão	Repo + branch secundária + pasta + DELETAR	Idem secundária
16	Excluir arquivo remoto EXATO	Exclusão	Repo, Branch, Pasta (RAIZ=ENTER), Arquivo	V1: caminho EXATO, sem varredura automática
17	Baixar	Transferência	Repo, Branch, Caminho remoto	mktemp + ln atômico, não sobrescreve
18	Enviar local	Transferência	Repo, Branch (ENTER=MAIN), Pasta destino, Arquivo local	Com atalho visual + cores
19	Clonar terceiro	Terceiros	URL HTTPS/SSH + destino	git clone seguro
20	Enviar anexo único	Upload	Repo, Branch (ENTER=MAIN), Pasta destino (ENTER=RAIZ)	V1: Branch+Pasta
21	Enviar múltiplos anexos	Upload	Repo, Branch, Pasta destino, lista anexos	V1: Branch+Pasta + resumo enviados/falhas
22	Criar repo MAIN	Criação	Nome, descrição, privado s/n	auto_init true
23	Criar branch secundária vazia	Criação	Repo + nome branch	Cria ref e esvazia
24	Criar pasta MAIN	Criação	Repo + caminho pasta	Cria .gitkeep
25	Criar pasta secundária	Criação	Repo + branch secundária + pasta	Idem
26	Renomear repo	Renomeação	Atual + novo	PATCH /repos
27	Renomear branch secundária	Renomeação	Atual + novo + RENOMEAR	Cria nova ref + deleta antiga
28	Renomear pasta	Renomeação	Repo, Branch, Atual, Novo, RENOMEAR	Reescreve árvore via git/trees
29	Mudar diretório local	Config	Novo diretório	Salva em ~/.config/github-manager/current-directory
30	Trocar usuário	Config	—	Limpa GITHUB_USER e TOKEN

10. Detalhes de Implementação das Funções Críticas V1

api(): valida output dentro de TMP_DIR, mkdir -p TMP_DIR, curl -sS -o output -w http_code com headers Accept, Authorization Bearer, X-GitHub-Api-Version. Trata curl_rc !=0, e http_code 200/201/202/204 sucesso, 401 token inválido, 403 rate limit, 404 não encontrado, 422 dados inválidos, 429 muitas requisições, com jq .message.
urlencode_path(): quebra por /, urlencode cada segmento via jq @uri, junta com /.
delete_remote_file() V1: pede repo, branch (ENTER=MAIN), mostra show_simple_repo_info, pede pasta (ENTER=RAIZ), pede arquivo com extensão, monta path = folder/file ou file se RAIZ, valida via api GET /contents/path?ref=branch, só então pede mensagem commit e confirma s/n, DELETE com sha e branch, audit_log DELETE_FILE_EXACT.
move_remote_files(): choose_repository_branch, show_simple_repo_info, pede origem e lista arquivos via remote_file_paths, pede nome do arquivo (com extensão), pede destino, monta destination_path = destino/relative, verifica destino 404, copia conteúdo via base64, PUT destino, DELETE origem, com mensagens coloridas.
move_all_files_bulk(): pede repo, branch, origem (RAIZ=ENTER), destino, lista via remote_file_paths, verifica truncated false, monta arrays paths/destinations, verifica colisão via associative array, preflight verifica destino 404 para todos, preview + confirma s/n, loop com GET origem, PUT destino, DELETE origem, com falha se arquivo > limite

Contents API.
upload_selected_attachments() V1: conta attachments_json via jq length, pede repo, pede branch (ENTER=MAIN) e valida via api GET /branches/{branch}, pede dest_folder (ENTER=RAIZ) e valida via validate_remote_folder, para cada file verifica -f e -r, monta remote_path = dest_folder/basename ou basename, base64, GET para sha, pergunta atualizar se existe, PUT com message, branch, sha, resumo sent/failed.
show_simple_repo_info(): imprime cabeçalho azul com repo e branch, api GET /git/trees/{branch}?recursive=1, lista pastas com jq .tree[] select type==tree, sort -u head 50, exibe em verde.
Cores: C_BG_BLUE+ C_WHITE para cabeçalhos, C_GREEN para pastas, C_YELLOW avisos, C_RED erros, C_CYAN separadores.

11. Permissões e Execução Global

Permissões: chmod +x install_gitdesktopV1.sh, chmod 700, ls -l deve mostrar -rwx------.
Global: sudo mv install_gitdesktopV1.sh /usr/local/bin/install_gitdesktopV1, sudo chmod 700 /usr/local/bin/install_gitdesktopV1, sudo chown $USER:$USER /usr/local/bin/install_gitdesktopV1, which install_gitdesktopV1. Depois, install_gitdesktopV1 de qualquer pasta, qualquer máquina.
Comando único que elimina antigo git_menu_CLI.sh e instala global: chmod 700 install_gitdesktopV1.sh && sudo rm -f /usr/local/bin/git_menu_CLI.sh ~/bin/git_menu_CLI.sh ./git_menu_CLI.sh ~/Desktop/*git_menu* ~/.local/share/applications/*git_menu* 2>/dev/null; sudo mv -f install_gitdesktopV1.sh /usr/local/bin/install_gitdesktopV1 && sudo chmod 700 /usr/local/bin/install_gitdesktopV1 && sudo chown $USER:$USER /usr/local/bin/install_gitdesktopV1 && which install_gitdesktopV1

12. Auditoria e Logs

- **audit.log:** XDG_STATE_HOME/github-manager/audit.log com data ISO, user, ação, detalhe. Ex: DELETE_FILE_EXACT, DELETE_FOLDER_MAIN, MOVE_FILE, UPLOAD.
- **ambiente.txt e pacotes-instalados.txt:** no backup, com distro, gerenciador, versões python/curl/jq/git e lista de pacotes (dpkg-query/rpm/pacman/apk).
- **TMP_DIR:** $TMPDIR/github-manager.XXXXXXXX, chmod 700, limpo no trap EXIT, com validação de caminho antes de rm -rf.

13. Licença Corporativa — Completa

LICENÇA CORPORATIVA — GITMENU Desktop V1 — install_gitdesktopV1.sh
Autoria: Everaldo Costa Silva — Especialista Linux — Bash/ksh, VMs, Containers, APIs

1. Objeto: Este software denominado GITMENU Desktop V1, arquivo install_gitdesktopV1.sh, é um instalador autônomo que contém todo o código Bash e Python do aplicativo GitHub Manager. É destinado a uso corporativo para gerenciamento de repositórios GitHub via API.
2. Propriedade Intelectual: Código de autoria de Everaldo Costa Silva. Todos os direitos reservados. A reprodução, distribuição ou modificação sem autorização formal por escrito é vedada.
3. Componentes de Terceiros: O software utiliza componentes de código aberto sob suas respectivas licenças, mantidas integralmente: Python 3 (PSF License), Tkinter/Tcl/Tk (Tcl/Tk License), Curl (MIT), Jq (MIT), Git (GPLv2), Zenity (LGPL), Coreutils/diffutils/findutils (GPLv3), Tar/Gzip (GPLv3). O uso desses componentes não altera a licença corporativa do conjunto.
4. Uso Autorizado: Uso interno e externo autorizado em máquinas Linux corporativas. É permitido instalar em /usr/local/bin e criar atalhos .desktop. Não é permitido revender como produto próprio sem aditivo contratual.
5. Isenção de Garantia: Fornecido "como está" (as is), sem garantias expressas ou implícitas de comercialização ou adequação a fim específico. O autor não se responsabiliza por perda de dados decorrente de uso indevido, embora o software implemente backup, rollback e validação de caminho exato.
6. Segurança e Credenciais: O PAT (Personal Access Token) do GitHub é de responsabilidade do usuário. O software não armazena PAT em disco, apenas em variável de sessão volátil, com logout_github() para limpeza. O usuário declara ciência de gerar PAT com escopos repo e delete_repo e de revogá-lo quando necessário.
7. Auditoria e Conformidade: Todas as operações destrutivas (exclusão de arquivo com caminho exato, exclusão de pasta, exclusão de repo/branch, movimentação) são registradas em audit.log com timestamp, usuário e caminho. O backup em ~/.local/state/github-manager-backups/ mantém rollback.sh validado contra symlink.
8. Suporte: Suporte via canal corporativo do autor, com base em audit.log, ambiente.txt e pacotes-instalados.txt do backup. Não há suporte para execução como root (bloqueado por EUID).
9. Vigência: Licença válida para a versão V1 a partir de 04/11/2026. Atualizações futuras (V2, etc.) seguirão novo termo.
10. Foro: Fica eleito o foro da comarca do autor para dirimir dúvidas.

14. Troubleshooting

- **401:** Token inválido/expirado — gere novo em github.com/settings/tokens.
- **403 rate limit:** aguarde, GitHub limita requisições.
- **404:** Repo/branch/pasta não existe — use atalho visual show_simple_repo_info para conferir nomes exatos.
- **422:** Dados inválidos — nome de branch/pasta com caracteres proibidos (//, \, ., ..).
- **Zenity não abre:** execute ~/.local/share/github-manager/github-manager.sh --task no terminal.
- **Rollback:** ~/.local/state/github-manager-backups/ULTIMO/rollback.sh
- **Permissões:** se ls -l não mostrar -rwx------, refaça chmod 700.

15. Changelog e Entrega

V1 Oficial — GITMENU Desktop V1 — install_gitdesktopV1.sh:
- Base com 3.040 linhas, 40+ funções, 30 tarefas.
- Cores ANSI + atalho visual show_simple_repo_info() em todas as telas de movimentação.
- Exclusão com caminho EXATO por pasta/RAIZ, sem varredura automática de nomes iguais.
- Upload de anexos (20/21) com Branch (ENTER=MAIN) + pasta destino (ENTER=RAIZ) + validação.
- Backup com rollback.sh validado, audit.log, segurança 700, symlink protection, PAT volátil.

Entrega: install_gitdesktopV1.sh + este PDF corporativo completo.